



TECHNICAL REPORT

Credit Risk ETL & Scoring Pipeline

An end-to-end PySpark pipeline that ingests a private-credit style loan portfolio (borrower → facility → transaction), engineers risk features with distributed window functions, and scores each facility's default risk using Spark ML.

Repository — github.com/jayantjain05/PIPELINE_CREDIT_RISK

Stack — PySpark 4.x, Spark ML, Python 3.13

Prepared — September 2026

1. Problem

Private credit lenders manage portfolios across multiple borrowers, each with one or more credit facilities, tracked through monthly payment and exposure snapshots. To monitor portfolio health, a lender needs to:

1. Reliably ingest and validate borrower / facility / transaction data
2. Derive standard credit risk features (delinquency buckets, vintage, utilization, payment behavior trends)
3. Score each facility's risk of near-term deterioration
4. Summarize risk at the portfolio level for reporting

This project implements that workflow end-to-end as distributed Spark transformations — ingestion, validation, feature engineering, and model training/scoring — rather than pandas/sklearn on a single machine dataframe, mirroring how this would be built for a real cluster-scale portfolio.

2. Data Model

Data is **synthetically generated** ([src/generate_synthetic_data.py](#)) to mimic a private-credit portfolio at borrower → facility → transaction grain, with realistic-ish payment behavior driven by a risk-tier-dependent default probability. A synthetic dataset was chosen over a public dataset (e.g. Lending Club) so the schema matches the borrower / facility / collateral / transaction structure used in real private-credit reporting, rather than a single flat loan table.

Borrower

borrower_id, industry, risk_tier (Prime / Near-Prime / Subprime), annual_revenue, years_in_business, onboarding_date

Facility

facility_id, borrower_id, facility_type (Term Loan / Revolver / ABL), credit_limit, interest_rate, disbursement_date, maturity_date, collateral_value

Transaction (monthly snapshot)

facility_id, period_end_date, scheduled_payment, actual_payment, outstanding_exposure, days_past_due

Generated volumes (this run)

2,000 borrowers · 2,954 facilities · 57,580 monthly transaction records (up to 24 months of history per facility)

3. Pipeline Architecture



Stage	File	What it does
1. Ingest	<code>ingest.py</code>	Reads raw CSVs with an explicit enforced schema; logs row counts, null rates, and duplicate counts per table
2. Clean & validate	<code>clean.py</code>	Dedupes on natural keys; enforces referential integrity (facilities reference a valid borrower, transactions a valid facility); quarantines failing records instead of silently dropping them
3. Feature engineering	<code>features.py</code>	Builds a facility-level snapshot with DPD buckets, vintage, utilization, and trailing-window features (roll-to-worse-bucket flag, 3-month avg payment ratio, 6-month max DPD)
4. Scoring	<code>score.py</code>	Trains Logistic Regression and Random Forest via <code>pyspark.ml.Pipeline</code> (StringIndexer → OneHotEncoder → VectorAssembler → classifier); evaluates with AUC and KS statistic
5. Output	<code>main.py</code>	Writes the feature table and scored portfolio to Parquet (partitioned by risk tier), plus a portfolio summary CSV

4. Key Design Decisions

- **Quarantine, don't drop.** Records failing validation are written to `data/processed/quarantine/` rather than discarded, so nothing disappears without a trace — mirrors how reconciliation would work in a real reporting pipeline.
- **Window functions for trailing behavior**, not row-by-row Python loops — `rolled_worse_3mo`, `payment_ratio_trailing3_avg`, and `dpd_max_trailing6` are computed with Spark `Window` partitioned by facility and ordered by period, the idiomatic distributed approach instead of collecting to a driver.
- **Random Forest vs. Logistic Regression** are both trained and compared, since Random Forest is expected to outperform Logistic Regression on this kind of tabular feature set; this pipeline confirms that pattern.
- **Label is a proxy** (60+ DPD or rolled to a worse bucket in the trailing 3 months), built directly from the same window features used to predict it — so the reported AUC/KS are **optimistic by construction** (see §7, Limitations).

5. Pipeline Run Results

Results below are from an actual execution of `python src/main.py` against the generated synthetic dataset.

2,954 FACILITIES SCORED	0 RECORDS QUARANTINED	9.3% HIGH-RISK RATE (TEST SET)	0.979 BEST MODEL AUC (RF)
-----------------------------------	---------------------------------	--	-------------------------------------

Ingest & Clean

Table	Row count	Valid	Quarantined
borrowers	2,000	2,000	0
facilities	2,954	2,954	0
transactions	57,580	57,580	0

Model comparison (held-out test set, 730 facilities)

Model	AUC	KS statistic
Logistic Regression	0.9755	0.8746
Random Forest (100 trees, depth 6)	0.9786	0.8977

AUC and KS are the standard scorecard evaluation metrics used in credit risk modeling. Random Forest modestly outperforms Logistic Regression here.

Risk tier breakdown (Random Forest scores, test set)

Risk tier	Facilities	Avg risk score	Avg utilization	High-risk count
Prime	338	0.028	17.7%	9
Near-Prime	279	0.103	18.1%	29
Subprime	113	0.282	26.7%	30

Portfolio summary — risk tier × DPD bucket (facility count)

Risk tier	Current	1–30	31–60	61–90	90+
Prime	329	8	1	0	0
Near-Prime	244	25	8	2	0
Subprime	70	25	9	1	8

6. Feature Engineering Detail

All features are computed per facility on the transaction history, then reduced to the latest snapshot per facility:

- **dpd_bucket** — Current / 1–30 / 31–60 / 61–90 / 90+, derived from `days_past_due`
- **vintage_months** — months between snapshot date and disbursement date
- **utilization_ratio** — `outstanding_exposure / credit_limit`
- **rolled_worse_3mo** — 1 if current DPD is worse than DPD three months prior (via `F.lag` over a facility-partitioned window)
- **payment_ratio_trailing3_avg** — trailing 3-month average of actual/scheduled payment (via a `rowsBetween(-2, 0)` window)
- **dpd_max_trailing6** — max DPD over the trailing 6 months (`rowsBetween(-5, 0)`)

The classifier pipeline (`score.py`) indexes and one-hot encodes `risk_tier` , `industry` , and `facility_type` , assembles them with the numeric features above into a single feature vector, and feeds that into `LogisticRegression` / `RandomForestClassifier` via a single `pyspark.ml.Pipeline` .

7. Limitations

Label leakage by construction. The target label is derived from the same trailing-window DPD features used as predictors, so the AUC/KS numbers in §5 demonstrate pipeline mechanics, not a validated forward-looking default model. A production version would use a strictly forward-looking label (e.g. "did this facility hit 90+ DPD in the *next* 3 months") with a proper train/test time split to avoid look-ahead bias.

- **Synthetic data** — payment behavior is simulated from a simple risk-tier-based probability, not real borrower behavior; it won't capture macro shocks, industry concentration effects, or seasonality.
- **Single-machine Spark** (`local[*]`) — this runs on one machine for development; a real cluster (EMR/Databricks) would need partition tuning and possibly a different file format strategy for the raw layer.
- **No orchestration tool** — `main.py` runs stages sequentially with logging. A production version would use Airflow/Dagster for scheduling, retries, and lineage.

8. Running It

```
pip install -r requirements.txt

# 1. Generate synthetic data (~2,000 borrowers, ~2,950 facilities, ~57,500 transactions)
python src/generate_synthetic_data.py

# 2. Run the full pipeline: ingest -> clean -> features -> score -> output
python src/main.py

# 3. Run tests
pytest tests/
```

Output lands in `data/processed/` :

- `feature_table/` — full engineered feature set (Parquet)
- `scored_portfolio/` — scored facilities, partitioned by risk tier (Parquet)
- `portfolio_summary/` — aggregated risk summary by tier × DPD bucket (CSV)
- `quarantine/` — any records that failed validation (CSV, if present)

Full source, tests, and this report live at github.com/jayantjain05/PIPELINE_CREDIT_RISK